Ders 7 Genişletilmiş Notu: Yapay Sinir Ağları: Perceptron, XOR Problemi ve Geri Yayılım

Ders 7 Genişletilmiş Notu: Yapay Sinir Ağları: Perceptron, XOR Problemi ve Geri Yayılım

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Biyolojik Nörondan Yapay Nörona
- 2. Biyolojik ve Yapay Nöronun Karşılıkları
- 3. Perceptron Modeli ve Doğrusal Ayırma
- 4. Hata Fonksiyonu ve Delta Kuralı
- 5. Öğrenme Oranı, Lokal ve Global Minimum
- 6. Aktivasyon Fonksiyonları
- 7. Eşik Değerinin Farklı Yazımları
- 8. Perceptron Öğrenme Algoritması
- 9. Örnek 1: AND Öğrenen Perceptron
- 10. Örnek 2: OR Öğrenen Perceptron
- 11. Epoch, İterasyon ve Regresyonla Fark
- 12. XOR Problemi
- 13. Çok Katmanlı Perceptron
- 14. Geri Yayılım (Backpropagation) Algoritması
- 15. XOR'un Öğrenilmesi, Momentum ve Hiperbolik Tanjant

1. Biyolojik Nörondan Yapay Nörona

Hoca derse yapay nöron ağlarının nereden çıktığını anlatarak başladı. Yapay zeka çalışmalarının hedefi, insanın beyni ve beş duyusuyla yaptığı işleri bilgisayara yaptırmaktır. En klasik örnekler parmak izi tanıma ve yüz tanımadır. Doğal dil işleme dersi alanlar ELIZA'yı da hatırlayacaktır: ELIZA, size sorular soran ve aldığı cevaplara göre yorum yapan bir "psikolog" programıdır. Bütün bu sistemlerin amacı, insanın gördüğü gibi görebilen, duyduğu gibi duyabilen ya da duyduğu cümleyi yorumlayabilen sistemler kurmaktır.

Başlangıçta kurulan sistemler matematiğe dayalıydı, ama insanın davrandığı gibi davranmıyordu. Hoca burada ilk derste söylediği bir şeyi hatırlattı: İnsanın asıl özelliği öğrenebilmesidir. Biz öğrendiklerimiz sayesinde tanıyabiliyor ve anlayabiliyoruz. Bu yüzden yapay zeka çalışmalarında kısa süre sonra yeni bir yol açıldı: İnsan beyninin öğrenme özelliğine sahip sistemler geliştirmek.

İnsan beyninin temel yapı taşı nörondur. Beyinde yaklaşık 10^{11} nöron vardır ve her nöron yaklaşık 10^4 nörona bağlıdır. Nöronlar öğrenmemizi, bilgiyi bellekte tutmamızı ve bir aktivite gerçekleştirmemizi sağlar. Her hücrenin bir aksonu vardır; akson hücredeki bilginin diğer hücrelere aktarılmasını sağlar. Hücreye gelen sinyaller dendritler üzerinden toplanır. Sinapslar her sinyalin ne kadar etkili olacağını, yani ağırlığını belirler. Gelen sinyaller ağırlıklı olarak toplanır. Bu ağırlıklı toplam belli bir eşik seviyesinin üzerindeyse aksona pozitif sinyal gönderilir ve sinyal diğer hücrelere aktarılır. Eşiğin altındaysa hücre çekinik kalır, bir işlem yapmaz. Soma ise hücrenin kendisidir (hücre gövdesi).

Yapay nöron çalışmaları bu yapıyı taklit etme fikriyle başladı: Madem insanın düşünmesini, karar vermesini, kolunu bacağını oynatmasını nöronlar sağlıyor, bu yapıyı yapay olarak kuralım. Hoca bu çalışmaların sanıldığı gibi yakın zamanda değil, 1940'larda başladığını vurguladı.

Beyindeki nöronlar birlikte çalışır. Beyin birbirine bağlı, paralel çalışabilen, dağıtık bir sistemdir. Hoca bunu şöyle örnekledi: Siz şu anda beni dinlerken aynı zamanda not alıyorsunuz; pencere açıksa ve hava soğuksa üşüdüğünüzü de düşünüyorsunuz. İnsan farkına varmadan aynı anda pek çok işi yapabiliyor. İlk yapay nöron çalışmalarının hedefi de buydu: çok sayıda, birbirine bağlı, paralel ve dağıtık çalışabilen nöronlar. Fakat donanımlar yetersiz olduğu için bu hedefe ulaşılamadı. Mevcut donanımla çalıştırılabilecek çok küçük modeller kullanıldı ve 2000'lerin başına kadar hep bu basit modellerle çalışıldı.

Sonra bir ekibin aklına şu soru geldi: Artık donanımlar çok güçlü, neden yapay nöron ağlarını ilk planlandığı gibi bütün gücüyle kullanmıyoruz? Buna bir de büyük veri eklendi. İnsanın en önemli özelliği, öğrendikçe genelleme yapabilmesidir. Hoca bunu küçük çocuk örneğiyle anlattı. Evde "anne" kavramını yeni öğrenen bir buçuk-iki yaşındaki çocuk, sokakta gördüğü bütün kadınlara "anne" diyebilir, çünkü gözünde bir anne modeli vardır. Zamanla daha çok insan gördükçe bir karakteristik çıkarmayı öğrenir ve başkalarına farklı hitap edildiğini anlar. Mantık budur: Bir şeyle ilgili ne kadar çok örnek görürsek o kadar iyi genelleme yapar, anlamı o kadar iyi anlarız. Derin öğrenme çalışmaları bu iki şeyden faydalanır: Yapay nöron ağlarının mimarisi ve çok büyük sayıda veri. Ağı çok büyük veriyle besleyerek insanın yaptığı gibi genelleme yapması sağlanır.

2. Biyolojik ve Yapay Nöronun Karşılıkları

Hoca slayttaki biyolojik nöron şekli üzerinden yapay nöronla karşılıkları tek tek gösterdi:

Biyolojik nöron	Yapay nöron	Görevi
Soma (hücre gövdesi)	Nöron (toplayıcı)	Gelen bilgiyi toplar
Dendrit	Giriş (input)	Giriş bilgilerini alır
Akson	Çıkış (output)	Toplanan bilgiyi dışarı (diğer nöronlara) iletir
Sinaps	Ağırlık (weight)	Gelen her bilginin ne kadar etkili olduğunu belirler

Hoca aynı slaytta ileride göreceğimiz çok katmanlı yapıyı da gösterdi: bir giriş katmanı (input layer), bir gizli katman (hidden layer) ve bir çıkış katmanı (output layer). Bu yapıya 13. bölümde dönülecek.

3. Perceptron Modeli ve Doğrusal Ayırma

En basit yapay nöron perceptron'dur. Yapay nöron mantığı McCulloch ve Pitts tarafından önerilmiştir (altyazıda "Mark Klok ve Pitts"). Perceptron'da bir toplayıcı ve ona bağlı girişler vardır.

Giriş x aslında bir vektördür: $x = (x_1, x_2, \dots, x_n)$. Bunlar, geçen hafta regresyonda gördüğümüz gibi özelliklerdir. Bir tanıma sisteminde kişinin yaşı, boyu, kilosu birer özellik olabilir. Tek boyutlu bir ses sinyali ya da iki boyutlu bir resim de özellik olarak verilebilir. Bilgiyi sisteme bir özellik vektörü olarak veririz. Bu bilgi önce toplanır, sonra çıkış bir eşik seviyesine bağlı olarak üretilir. Çıkışın aktif mi pasif mi olduğunu gösteren fonksiyonlara aktivasyon fonksiyonu (activation function) denir.

Bütün özellikler aynı etkide değildir. Her özelliğe bağlı bir ağırlık vardır ve ağırlık o özelliğin etkisini belirler. Hoca ağırlıkları v ile gösterdi. Ağırlıklı toplam, giriş vektörü ile ağırlık vektörünün iç çarpımıdır (dot product):

$$\sum_{i=1}^{n} v_i x_i = v_1 x_1 + v_2 x_2 + \cdots + v_n x_n$$

Yani her özellik kendi ağırlığıyla çarpılır ve sonuçlar toplanır. Bizim için önemli olan, bu toplamın bir aktivasyon fonksiyonundan geçirilmesidir. Örneğin toplam 0'dan büyükse $y = 1$, değilse $y = -1$ diyebiliriz. Toplamı $X = \sum_i v_i x_i$ diye adlandırırsak ve bir eşik (threshold) θ kullanırsak:

$$y = \begin{cases} +1, & X \geq \theta \\ -1, & X < \theta \end{cases}$$

Temel perceptron modeli budur. Buradaki aktivasyon fonksiyonu sign fonksiyonudur: eşikten büyükse $+1$, küçükse -1. Eşik, v_0 diye ayrı bir ağırlık gibi de yazılabilir:

$$y = \text{sign}\left(\sum_{i=1}^{n} v_i x_i + v_0\right)$$

Böylece iki sınıflı bir sınıflandırıcı elde ederiz: $y = +1$ ise örnek A sınıfındadır, $y = -1$ ise B sınıfındadır.

3.1 İki Boyutta Karar Sınırı

İki boyutlu uzayda göstermek her zaman daha kolaydır. Özelliklerimiz x_1 ve x_2 olsun. $v_1 x_1 + v_2 x_2 + v_0 = 0$ doğrusu düzlemi ikiye ayırır. Doğrunun üstü $y = 1$ bölgesi, altı $y = -1$ bölgesidir. Bir tarafta A sınıfına, diğer tarafta B sınıfına ait örnekler varsa, perceptron bu örnekleri doğrusal (lineer) olarak iki sınıfa ayırmış olur.

İki sınıfı birbirinden ayıran bu sınıra hiper düzlem (hyperplane) denir. Toplamın sıfıra eşit olduğu yer hiper düzlemdir; bir tarafında toplam sıfırdan büyük, diğer tarafında sıfırdan küçüktür. Perceptron'un yapabildiği şey doğrusal ayrılabilen (linearly separable) sınıfları ayırmaktır. İkiden fazla sınıf da olabilir; önemli olan sınıfların doğrusal olarak ayrılabilmesidir. Hoca çok sınıf konusundan daha sonra bahsedeceğini söyledi; bu derste iki sınıf ($+1$ ve -1) inceleniyor.

Soru: Bias ile threshold burada aynı anlama mı geliyor? **Cevap:** Evet, aynı şey. Regresyonda da koordinat sisteminde kaydırmak için bias kullanmıştık. Burada da v_0 bir eşik seviyesini gösteriyor ve çıkışın ne olacağı ona bağlı olarak belirleniyor.

Soru: Bu doğru orijinden geçmiyor, değil mi? **Cevap:** Hayır, orijinden geçmesi gerekmiyor.

> **Ek bilgi (derste anlatılmadı):** Bias ile eşik aynı rolü oynar, ama işaretleri terstir. $\sum v_i x_i \geq \theta$ koşulu $\sum v_i x_i - \theta \geq 0$ demektir; yani bias $b = -\theta$ olarak yazılır. Kaynaklarda ikisi de "eşik" ya da "bias" diye geçebilir.

4. Hata Fonksiyonu ve Delta Kuralı

Hata hesabı doğrusal regresyondakiyle aynıdır. Bir maliyet fonksiyonumuz (cost function) vardır. Maliyet fonksiyonu, hesapladığımız çıkışın istenen çıkışa ne kadar yakın olduğunu ölçer. j. örnek için istenen çıkış $y_{d,j}$, hesaplanan çıkış y_j ise:

$$E = \frac{1}{2} \sum_j \left(y_{d,j} - y_j\right)^2$$

Buna hata kareleri toplamı (sum of squared errors) denir. Kare alınmasının iki nedeni vardır: hatanın pozitif olması ve büyük hataların daha da büyütülmesi. Başa konan $\frac{1}{2}$ türev alırken kolaylık sağlar; karenin türevinden gelen 2 ile sadeleşir.

Hesaplanan çıkış ağırlıklara bağlıdır: $y_j = \sum_i v_i x_{i,j}$. Hatanın ağırlığa göre türevi alınınca ağırlığın nasıl değişmesi gerektiği bulunur:

$$\frac{\partial E}{\partial v_i} = -\sum_j \left(y_{d,j} - y_j\right) x_{i,j}$$

Hoca bu türev kısmını hızlı geçti; amacının formülün nereden geldiğini göstermek olduğunu, ayrıntının çok önemli olmadığını söyledi. Önemli olan sonuçtur: Ağırlıktaki değişim "istenen çıkış eksi hesaplanan çıkış" çarpı "giriş" ile orantılıdır.

Gradyan inişinin (gradient descent) mantığı geçen dersten bilinir. Yatay eksende ağırlık, dikey eksende hata olsun. Ağırlık başta rastgele bir değerle belirlenir ve iteratif olarak değişir. İlk ağırlık değerleri için hata çok büyüktür. Ağırlık yavaş yavaş değiştirilerek global minimuma, yani hatanın en az olduğu yere inilmek istenir.

Bunun için delta kuralı (delta rule) kullanılır. Hoca bu kuralın Widrow-Hoff kuralı olarak da bilindiğini söyledi (altyazıda "Hofloin"). t anındaki hata $e(t) = y_d(t) - y(t)$ olmak üzere, ağırlık değişimi ve yeni ağırlık:

$$\Delta v_i(t) = \alpha \cdot e(t) \cdot x_i, \qquad v_i(t+1) = v_i(t) + \Delta v_i(t)$$

Burada α öğrenme oranıdır (learning rate). Gradyan inişinin sıkıntısı, ne kadar büyük adımla ilerleneceğine karar vermektir; öğrenme oranı bu adımı belirler. Kısaca $t+1$ anındaki i. özelliğin ağırlığı, t anındaki ağırlık artı öğrenme oranı çarpı hata çarpı giriştir.

5. Öğrenme Oranı, Lokal ve Global Minimum

Soru: Learning rate pozitif mi olacak? **Cevap:** Evet. Learning rate 0 ile 1 arasında değişen bir orandır. 0'a yaklaştıkça öğrenme yavaşlar, 1'e yaklaştıkça hızlanır.

Ağırlığa göre hatanın değişimi düzgün bir eğri değildir. Bazı yerlerde hata azalıyor gibi görünür, sonra bir ağırlık değerinden sonra tekrar yükselebilir. Eğrideki bu çukurlardan biri lokal minimum (local minimum), en derini ise global minimumdur. Yapılması gereken iki şey vardır:

- Lokal minimuma takılmamak.
- Global minimumu atlayıp geçmemek.

Atlayıp geçmek şu demektir: Ağırlığı büyük adımlarla değiştirirseniz global minimumun üzerinden atlayıp öbür tarafa geçebilirsiniz. Bunu önlemek için öğrenme oranı kullanılır. İşlem iteratif olduğu için öğrenme oranı başta büyük alınır. Hatanın büyük olduğu yerlerde yavaş ilerlemenin anlamı yoktur; ağırlık hızlı değişsin. Belli bir iterasyondan sonra öğrenme oranı küçültülür. Öğrenme oranı küçükse ağırlığın değişimi yavaş olur.

Hoca bunu sayılarla anlattı: Öğrenme oranını 0,1 alırsanız hata çarpı giriş değeri 0,1 ile çarpılır ve ağırlık $t+1$ anında yavaş değişir. 0,9 alırsanız bu çarpım çok etkili olur ve ağırlık büyük değişir. Bu yüzden iterasyonların başında ağırlık büyük büyük, ilerledikçe yavaş yavaş değiştirilir. Böylece minimuma yaklaşıldığında ağırlık yavaşça iner ve minimumu kaçırıp başka bir ağırlık noktasına atlamaz.

6. Aktivasyon Fonksiyonları

Hoca şimdiye kadar sign fonksiyonunu kullandığımızı hatırlattı ve diğer aktivasyon fonksiyonlarını gösterdi. Hepsinde girişe verilen değer, eşik seviyesi de dahil olmak üzere ağırlıklı toplamdır: $v_0 + v_1 x_1 + v_2 x_2$.

- **Step fonksiyonu:** Toplam 0'dan büyükse (veya eşitse) çıkış 1, küçükse 0.

$$y = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- **Sign fonksiyonu:** Toplam 0'dan büyükse (veya eşitse) $+1$, küçükse -1. Ani bir geçiş sağlar: çıkış ya -1 ya $+1$ olur.

$$y = \begin{cases} +1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

- **Sigmoid fonksiyonu:** Geçen derste lojistik regresyonda görülmüştü. S harfine benzer ve yumuşak bir geçiş sağlar.

$$y = \frac{1}{1 + e^{-x}}$$

- **Doğrusal (lineer) fonksiyon:** Çıkış, girişe bağlı olarak doğrusal olur ($y = x$).

Hangisinin ne zaman kullanıldığı: Tek katmanlı perceptron (single-layer perceptron), yani bir nöron, ona gelen girişler ve bir çıkıştan oluşan yapı, genelde sign (ya da step) fonksiyonu kullanır. Lineer fonksiyon çok kullanılmaz. Sigmoid ise çok katmanlı perceptron'da gerekecektir.

Perceptron 1958'de Rosenblatt tarafından önerildi; yapay nöron ağının öğrenebildiğinin ilk gösterimi budur. Rosenblatt'ın perceptron'u, McCulloch ve Pitts'in yapay nöron modeline dayanır: iki giriş, girişlerin ağırlıklı toplamı ve bir eşikten geçirilerek üretilen y çıkışı.

> **Ek bilgi (derste anlatılmadı):** Sigmoid'in birkaç değeri: $\sigma(0) = 0{,}5$, $\sigma(2) \approx 0{,}88$, $\sigma(-2) \approx 0{,}12$. Çıkış her zaman 0 ile 1 arasındadır; negatif değer üretmez. Türevi kendi çıkışıyla yazılabilir: $\sigma'(x) = \sigma(x)\,[1 - \sigma(x)]$. Bu özellik 14. bölümdeki geri yayılım formüllerinde $y(1-y)$ çarpanı olarak karşımıza çıkar.

7. Eşik Değerinin Farklı Yazımları

Aradan sonra bir öğrencinin sorusu üzerine hoca, eşik değerinin kaynaklarda farklı biçimlerde yazıldığını ayrıntılı anlattı. Temel kural hep aynıdır: Gerçek girişlerin ve bu girişlere bağlı ağırlıkların iç çarpımı bir eşik seviyesinden büyükse bir aktivasyon olur, küçükse başka bir aktivasyon olur. Bunu kod yazarken kolaylık olsun diye farklı biçimlerde görebilirsiniz:

1. **Eşikle karşılaştırma:** $\sum_{i=1}^{n} v_i x_i > \theta$ ise bir sınıf, değilse diğer sınıf.
2. **Eşikten çıkarma:** $\sum_{i=1}^{n} v_i x_i - \theta$ hesaplanır ve sonucun işaretine bakılır.
3. **Eşiği ağırlık gibi ekleme:** x_0 diye her zaman sabit bir giriş eklenir ve eşik onun ağırlığı v_0 olur:

$$\sum_{i=0}^{n} v_i x_i = x_0 v_0 + x_1 v_1 + \cdots + x_n v_n$$

Üçüncü biçimde x_0 bazı kaynaklarda -1, bazılarında $+1$ alınır. $x_0 = -1$ alınırsa toplamın içine $-v_0$ girer, yani v_0 doğrudan eşik olur. $x_0 = +1$ alınırsa $+v_0$ girer. Değişen şey ağırlığın işareti olduğu için sonuç etkilenmez. Hoca şunu özellikle vurguladı: x_1, x_2, x_3 gerçek özelliklerdir, ama x_0 bir özellik değildir. x_0 eşiğe bağlı, sabit bir giriş değeridir.

Eşiğin güncellenmesinde de varyasyonlar vardır:

- Bazı kaynaklarda eşik sabit tutulur, yalnız ağırlıklar değişir.
- Bazılarında eşik de bir ağırlık gibi güncellenir.

> **Hocanın vurgusu:** Bunlar varyasyonlardır. Farklı dokümanlarda hepsini görebilirsiniz. Farklı bir biçim görünce "bu niye böyle oldu" diye düşünmeyin; matematiksel karşılığından anlayacağınız gibi hepsi aynı kapıya çıkar. Hepsinde iki çıkış üretilir ve üretilen çıkışa bakarak örneğin A sınıfına mı B sınıfına mı ait olduğuna karar verilir.

Soru: Eşitlik ("büyük eşittir" ya da "küçük eşittir") hangi tarafta olursa olsun sonuç değişmiyor mu? 1 ile 0 arasında fark var. **Cevap:** O bir anlık bir durumdur. Toplam tam hiper düzlemin üzerine, yani tam eşiğe denk geldiğinde olur. Ağırlığı biraz oynattığınız anda örnek ya bir tarafa ya öbür tarafa kaçar. İterasyonun devamında hangi tarafta olması doğruysa ağırlık ona göre azalır ya da artar ve yerine yerleşir. Zaten bir örneğin hiper düzleme çok yakın olması iyi bir şey değildir; hiper düzlemden ne kadar uzaksa o kadar iyidir. Bu yüzden kimi kaynak "büyüktür sıfır", kimi "büyük eşittir sıfır" yazar; buna çok takılmamak gerekir.

8. Perceptron Öğrenme Algoritması

Hoca öğrenmenin ne demek olduğunu şöyle özetledi: Öğrenme tamamen ağırlıkların değişmesidir. Girişleriniz ve istenen çıkışlarınız sabit bilgilerdir. Öğrenmeyi ancak ağırlıkları değiştirerek sağlayabilirsiniz. Aynı ağ bir ağırlık takımıyla AND, başka bir ağırlık takımıyla OR işlemi

yapabilir. Verilen girişler için istenen çıkışlar elde edilince ağırlıklar yerleşmiş olur. Böylece bilinmeyen bir örnekte de istenen çıkışın elde edilmesi sağlanır.

Algoritma dört adımdan oluşur (eşik v_0 olarak ağırlıklar gibi güncellenecek şekilde):

1. **Başlatma (initialization):** Ağırlıklar genellikle $-0{,}5$ ile $+0{,}5$ arasında rastgele değerlerle başlatılır.
2. **Aktivasyon:** t anındaki çıkış hesaplanır: $y(t) = \text{sign}\left(\sum_{i=0}^{n} x_i v_i\right)$.
3. **Ağırlık eğitimi (weight training):** Hata ve ağırlık değişimi hesaplanır, ağırlıklar güncellenir:

$$e(t) = y_d(t) - y(t), \qquad \Delta v_i(t) = \alpha \cdot e(t) \cdot x_i, \qquad v_i(t+1) = v_i(t) + \Delta v_i(t)$$

4. **İterasyon:** Yakınsama (convergence) sağlanana kadar 2. adıma dönülür. Yakınsama, hatanın belirlenen bir eşik seviyesinin altına düşmesidir.

Hoca 3. adımdaki işaretin anlamını ayrıntılı açıkladı. Giriş değeri x_i zamanla değişmez; her örnek için neyse odur. Hata ise istenen çıkış eksi hesaplanan çıkıştır:

- **Gerçek (istenen) çıkış hesaplanandan büyükse** hata sıfırdan büyüktür. Pozitif bir Δv oluşur ve $t+1$ anındaki ağırlık artar. Bu da y çıkışının büyümesini ve hedefe (target) yaklaşmasını sağlar.
- **İstenen çıkış hesaplanandan küçükse** hata negatiftir. Δv negatif olur ve ağırlık azalır. Böylece y çıkışı hedefe yaklaşacak şekilde küçülür.

Ağırlık eğitiminde yapılan şey, ağırlıkların istenen çıkışı elde edebilecek şekilde azaltılması ya da artırılmasıdır.

9. Örnek 1: AND Öğrenen Perceptron

Hoca slayttaki tablo üzerinden AND işlemini öğrenen perceptron'u anlattı. İki giriş (x_1, x_2) ve bir çıkış (y) vardır. AND doğruluk tablosu:

x_1	x_2	y_d
0	0	0
0	1	0
1	0	0
1	1	1

Bu örnekte şunlara dikkat etmek gerekir:

- Eşik **sabit** alınmıştır: $\theta = 0{,}2$. Eşik ayrı bir ağırlık olarak güncellenmez; yalnız ağırlıklar değişir.
- Aktivasyon olarak **step** fonksiyonu kullanılır: toplam eşikten büyük veya eşitse çıkış 1, değilse 0.
- Başlangıç ağırlıkları rastgele seçilmiştir: $v_1 = 0{,}3$, $v_2 = -0{,}1$. Öğrenme oranı $\alpha = 0{,}1$.

Hoca bir epoch'un (altyazıda "ipok") bir tur, yani dört eğitim örneğinin hepsinin bir kez gösterilmesi olduğunu hatırlattı. Çözüm sırasında kullanılan iki formül: $e = y_d - y$ ve $\Delta v_i = \alpha \cdot e \cdot x_i$.

1. epoch, adım adım:

- **Giriş** $(0,0)$**:** Toplam $0 \cdot 0{,}3 + 0 \cdot (-0{,}1) = 0$. $0 < 0{,}2$ olduğu için $y = 0$. İstenen çıkış da 0, hata 0. Ağırlıklar değişmez.
- **Giriş** $(0,1)$**:** Toplam $0 \cdot 0{,}3 + 1 \cdot (-0{,}1) = -0{,}1$. $-0{,}1 < 0{,}2$ olduğu için $y = 0$. İstenen çıkış 0, hata 0. $\Delta v_1 = 0{,}1 \cdot 0 \cdot 0 = 0$, $\Delta v_2 = 0{,}1 \cdot 0 \cdot 1 = 0$. Ağırlıklar değişmez: $v_1 = 0{,}3$, $v_2 = -0{,}1$.
- **Giriş** $(1,0)$**:** Toplam $1 \cdot 0{,}3 + 0 \cdot (-0{,}1) = 0{,}3$. $0{,}3 > 0{,}2$ olduğu için $y = 1$. Ama bu giriş için istenen çıkış 0'dır. Hata $e = 0 - 1 = -1$.
  - $\Delta v_1 = 0{,}1 \cdot (-1) \cdot 1 = -0{,}1$, yani $v_1 = 0{,}3 - 0{,}1 = 0{,}2$.
  - $\Delta v_2 = 0{,}1 \cdot (-1) \cdot 0 = 0$, yani $v_2 = -0{,}1$ kalır.

- **Giriş** $(1,1)$**:** Toplam $1 \cdot 0{,}2 + 1 \cdot (-0{,}1) = 0{,}1$. $0{,}1 < 0{,}2$ olduğu için $y = 0$. İstenen çıkış 1, hata $e = 1 - 0 = 1$.
  - $\Delta v_1 = 0{,}1 \cdot 1 \cdot 1 = 0{,}1$, yani $v_1 = 0{,}2 + 0{,}1 = 0{,}3$.
  - $\Delta v_2 = 0{,}1 \cdot 1 \cdot 1 = 0{,}1$, yani $v_2 = -0{,}1 + 0{,}1 = 0$.

Hoca bu tabloyu anlatırken birkaç kez yanlış değer yazıp düzeltti (v_2'yi önce $0{,}1$ yazdı, son adımda v_1'i önce $0{,}3$ aldı) ve öğrencilerden hata yaptığında kendisini uyarmalarını istedi. Yukarıdaki değerler düzeltilmiş hâlidir.

Bu tur anlatılan mantığın özeti şudur: İlk iki adımda hata 0 olduğu için ağırlıklar değişmedi. Hata 0 ise ağırlık değiştirilmez. Üçüncü adımda hata -1 oldu: hesaplanan çıkış istenenden büyük, demek ki ağırlığı azaltmak gerekiyor; v_1 0,2'ye düştü. Son adımda istenen çıkış 1 iken hesaplanan 0 çıktı, hata $+1$ oldu; bu sefer ağırlıklar büyüdü, çünkü hesaplanan çıkışın istenene yaklaşması için ağırlıkların artması gerekir. Birinci turun sonunda ağırlıklar $v_1 = 0{,}3$, $v_2 = 0$ oldu.

Mantık regresyondakinin aynısıdır: Aynı örnekler için tur tur dönülür. Eğitim kümesinde dört örnek olduğu için yeni ağırlıklarla aynı dört örnek için yeni bir tur dönülür. Slayttaki tabloya göre ağ dört turda hâlâ hata yapıyordu; **beşinci turda hata tamamen sıfır oldu**. Beşinci turun ağırlıkları $v_1 = 0{,}1$, $v_2 = 0{,}1$, eşik $\theta = 0{,}2$:

- $(0,0)$: $0 < 0{,}2 \Rightarrow y = 0$
- $(0,1)$: $0{,}1 < 0{,}2 \Rightarrow y = 0$
- $(1,0)$: $0{,}1 < 0{,}2 \Rightarrow y = 0$
- $(1,1)$: $0{,}1 + 0{,}1 = 0{,}2 \geq 0{,}2 \Rightarrow y = 1$

Toplam, eşik seviyesine eşit olduğu için son girişte çıkış 1 olur. Bu turda dört örneğin hiçbirinde hata olmadığı için öğrenme kesilebilir. Ağ AND'i öğrenmiştir.

> **Ek bilgi (derste anlatılmadı):** Tablonun tamamı python ile yeniden hesaplandı. Her turun sonundaki ağırlıklar şöyledir: 1. tur $(0{,}3;\ 0)$, 2. tur $(0{,}2;\ 0)$, 3. tur $(0{,}2;\ 0{,}1)$, 4. tur $(0{,}1;\ 0{,}1)$. Beşinci turda hiç hata çıkmaz. Bu, hocanın "dört turda hâlâ hata vardı, beşinci turda sıfır oldu" sözüyle uyumludur.

Aradan sonra bir öğrenci aynı algoritmayı Excel'de kurduğunu ve beşinci iterasyonda doğru sonucu verdiğini söyledi. Öğrencinin sorusu, bulunan doğrunun ($0{,}1x + 0{,}1y = 0$) neden sınıfları ayırmadığıydı. Hoca eşiğin de hesaba katılması gerektiğini hatırlattı: Karar sınırı $x_1 v_1 + x_2 v_2 - \theta = 0$ doğrusudur. Öğrenci eşiği dikkate almadığını fark etti.

> **Ek bilgi (derste anlatılmadı):** Eşik katılınca karar sınırı $0{,}1x_1 + 0{,}1x_2 - 0{,}2 = 0$, yani $x_1 + x_2 = 2$ doğrusu olur. Bu doğru yalnız $(1,1)$ noktasından geçer; $(0,0)$, $(0,1)$ ve $(1,0)$ doğrunun altında kalır. Eşitlik durumunda çıkış 1 kabul edildiği için $(1,1)$ 1 sınıfına düşer.

10. Örnek 2: OR Öğrenen Perceptron

Öğrencilerin isteği üzerine hoca tahtada farklı bir örnek çözdü. Elle hesap uzun sürmesin diye eşiği ayrı ve sabit tuttu: $\theta = 0{,}3$. Hoca önce yazdığı tablonun AND olduğunu bir öğrencinin uyarısıyla fark etti ve OR tablosuyla devam etti. Öğrencilere step ya da sign fonksiyonundan hangisini tercih ettiklerini sordu; step fonksiyonu seçildi.

OR doğruluk tablosu: $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 1$.

Başlangıç: $v_1 = 0{,}2$, $v_2 = 0{,}2$, $\theta = 0{,}3$ (sabit), $\alpha = 0{,}3$. Hoca değerleri, ağın hemen öğrenmesin, birkaç adım görülsün diye seçtiğini söyledi.

1. epoch:

- **Giriş** $(0,0)$**:** Toplam 0. $0 < 0{,}3$ olduğu için $y = 0$. Hata 0, değişiklik yok.
- **Giriş** $(0,1)$**:** Toplam $0 \cdot 0{,}2 + 1 \cdot 0{,}2 = 0{,}2$. $0{,}2 < 0{,}3$ olduğu için $y = 0$. İstenen çıkış 1, hata $e = 1$.
  - $v_1 = 0{,}2 + 0{,}3 \cdot 1 \cdot 0 = 0{,}2$ (değişmedi).
  - $v_2 = 0{,}2 + 0{,}3 \cdot 1 \cdot 1 = 0{,}5$.
- **Giriş** $(1,0)$**:** Toplam $1 \cdot 0{,}2 + 0 \cdot 0{,}5 = 0{,}2$. $0{,}2 < 0{,}3$ olduğu için $y = 0$. İstenen çıkış 1, hata $e = 1$.

  - $v_1 = 0{,}2 + 0{,}3 \cdot 1 \cdot 1 = 0{,}5$.
  - $x_2 = 0$ olduğu için $v_2 = 0{,}5$ kalır.
- **Giriş** $(1, 1)$**:** Toplam $1 \cdot 0{,}5 + 1 \cdot 0{,}5 = 1$. $1 > 0{,}3$ olduğu için $y = 1$. Hata 0, ağırlıklar aynı kalır: $v_1 = 0{,}5$, $v_2 = 0{,}5$.

2. epoch ($v_1 = 0{,}5$, $v_2 = 0{,}5$, $\theta = 0{,}3$):

- $(0, 0)$: $0 < 0{,}3 \Rightarrow y = 0$ (doğru)
- $(0, 1)$: $0{,}5 > 0{,}3 \Rightarrow y = 1$ (doğru)
- $(1, 0)$: $0{,}5 > 0{,}3 \Rightarrow y = 1$ (doğru)
- $(1, 1)$: $0{,}5 + 0{,}5 = 1 > 0{,}3 \Rightarrow y = 1$ (doğru)

Bu turun sonunda toplam hataya bakıldığında hiç hata yoktur. Devam etmeye gerek kalmaz; öğrenme tamamlanmıştır. Ağ OR'u öğrenmiştir. (Bu iki tur python ile yeniden hesaplandı; sonuç aynıdır.)

Hoca her adımda yapılanları şöyle özetledi: Her adımda önce çıkış hesaplanır. Çıkışa bağlı olarak hata, hataya bağlı olarak da ağırlığın nasıl değişeceği hesaplanır.

11. Epoch, İterasyon ve Regresyonla Fark

Örneklerden sonra öğrenciler birkaç soru sordu.

Soru: Örnekte $-0{,}2$ bulduk. Neden bu değerden istenen çıkışı çıkarmadık? Slayttaki tabloda y_d (istenen çıkış) vardı. **Cevap:** $-0{,}2$ hesaplanan toplamdır, çıkış değil. Toplamın kendisi hiçbir zaman hata hesabında kullanılmaz. Önce toplamın eşiğe göre durumuna bakılır: eşikten büyükse bir çıkış, küçükse başka bir çıkış üretilir (step fonksiyonunda 1 ya da 0, sign fonksiyonunda $+1$ ya da -1). Hata, bu üretilen çıkış ile istenen çıkış arasındaki farktır. Mutlaka bir aktivasyon olması için eşik vardır. Öğrenci başta yanlış anladığını söyledi.

Soru: Epoch ile iterasyon arasındaki fark nedir? Örneğin 5 tur ve 4 örnek için 20 iterasyon, 5 epoch mu oldu? **Cevap:** Evet. Diyelim ki 30 eğitim örneğiniz var. 30 örneğin hepsini gösterdiğinizde bir epoch tamamlanır. Her örneği gösterdiğinizde ağırlıkları değiştirirsiniz ve sonunda toplam hataya bakarsınız. Bir örnekte hata oluşmaması öğrenmenin bittiği anlamına gelmez. Bütün örnekler gösterildiğinde hata kalmıyorsa, ya da hata belirlenen minimum değerin altındaysa öğrenme tamamlanmıştır. İterasyon ise tek bir örneği göstermektir: $t = 3$ anı, $t = 4$ anı gibi; zaman adımı, kaçıncı örneğin gösterildiğidir.

Soru: Problemin sınıflandırma ya da regresyon olması bu adımlarda neyi değiştiriyor? Step fonksiyonu seçimi mi? **Cevap:** İkisinde de öğrenme ağırlıkların değişmesiyle olur ve ikisinde de gradyan inişi kullanılır; bu yüzden benzer görünürler. Girişleri ya da çıkışları değiştiremezsiniz, değişen ağırlıklardır. Fark şudur: Regresyonda gerçek değerli bir çıkış üretilir ve bu çıkış doğrudan kullanılır. Sınıflandırmada bir eşik mantığı vardır: Hesaplanan çıkış eşikten büyükse bir sınıf, değilse diğer sınıf denir; sınıf $+1/-1$ ya da $1/0$ ile gösterilir. Eşiğin oluşturduğu çizgi karar sınırıdır (decision boundary). Karar sınırının bir tarafı bir sınıfı, öbür tarafı diğer sınıfı gösterir. Perceptron'da öğrenmek, karar sınırını nereye koyacağınıza karar vermektir.

Hoca karar sınırının yeri konusunda bir de şunu ekledi: İki sınıfı doğru ayıran birden çok karar sınırı çizilebilir. "İyi öğrenme" ile "daha iyi öğrenme" arasındaki fark karar sınırının yerinden gelir. Bu konu destek vektör makinelerinde (SVM) ayrıca görülecek. Regresyonda ise çıkış $0{,}4$, $0{,}7$ gibi gerçek bir sayıdır; veri normalize edildiyse 0 ile 1 arasında kalabilir ve sonra gerçek ölçeğe geri dönüştürülür.

Soru: Regresyonla ilgili küçük bir örnek çözebilir misiniz? **Cevap:** Hoca bu dersin konusunun regresyon olmadığını, hazır bir regresyon örneği olmadığını söyledi. Mantığı anlarsanız regresyon örneğini de elle çözebilirsiniz; bulursa bir örnek çözeceğini ekledi.

Soru: Regresyon yapmak istersek aktivasyon fonksiyonu mu değişiyor? **Cevap:** Regresyonda aktivasyon fonksiyonu yoktur; çıkış doğrudan kullanılır. Aktivasyon fonksiyonunu lojistik regresyonda (sınıflandırma için) kullanıyorsunuz.

12. XOR Problemi

İkinci aradan sonra hoca çok katmanlı perceptron'un neden gerektiğini anlattı. Tek katmanlı perceptron sınıfları doğrusal olarak ayırır. İki özellik için hiper düzlem $x_1v_1 + x_2v_2 - \theta = 0$ doğrusudur. $x_1v_1 + x_2v_2 - \theta$ değerinin sıfırdan büyük olduğu taraf bir sınıfı, küçük olduğu taraf öbür sınıfı gösterir.

Hoca bunu x_1-x_2 düzleminde $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$ noktaları üzerinde çizdi:

- **AND'de** yalnız $(1,1)$ noktasının çıkışı 1'dir. Bu noktayı diğer üçünden ayıran bir doğru çizilebilir: doğrunun bir tarafı çıkışın 1 olduğu A sınıfı, öbür tarafı çıkışın 0 olduğu B sınıfıdır.
- **OR'da** yalnız $(0,0)$ noktasının çıkışı 0'dır. Bu noktayı diğer üçünden ayıran bir doğru da çizilebilir.

XOR (altyazıda "egzor") tablosu ise şöyledir:

x_1	x_2	y_d
0	0	0
0	1	1
1	0	1
1	1	0

Girişler aynıyken çıkış 0, girişler farklıyken çıkış 1'dir. Düzlemde $(0,0)$ ve $(1,1)$ bir sınıfta, $(0,1)$ ve $(1,0)$ diğer sınıftadır. Bu sınıflar çapraz durduğu için onları tek bir doğruyla ayırmak mümkün değildir.

Hoca durumu matematiksel olarak da gösterdi. Çıkışın 1 olması için toplamın eşikten büyük, 0 olması için küçük olması gerekir:

- $(0,0) \to 0$: $0 < \theta$, yani eşik pozitif olmalıdır.
- $(0,1) \to 1$: $x_2v_2 = v_2 > \theta$ olmalıdır.
- $(1,0) \to 1$: $x_1v_1 = v_1 > \theta$ olmalıdır.
- $(1,1) \to 0$: $v_1 + v_2 < \theta$ olmalıdır.

Bir taraftan v_1 ve v_2'nin tek başına eşikten büyük olması, öbür taraftan ikisinin toplamının eşikten küçük olması gerekiyor. Eşik pozitif olduğu için bu mümkün değildir: $v_1 > \theta > 0$ ve $v_2 > \theta > 0$ ise $v_1 + v_2 > 2\theta > \theta$ olur. Bu yüzden tek katmanlı perceptron XOR'u öğrenemez.

> **Hocanın vurgusu:** Doğrusal ayrılamayan sınıflandırma problemleri için çok katmanlı perceptron (multi-layer perceptron, MLP) kullanılır.

Hoca tek yapay nöron modelinin perceptron olmadığını, başka birçok modelin de olduğunu, ders boyunca birkaçını göstereceğini söyledi. MLP ise en temel ve derin öğrenmede de en çok kullanılan yapay nöron ağı mimarisidir.

13. Çok Katmanlı Perceptron

Hoca XOR için bir MLP çizdi: x_1 ve x_2 girişleri, araya eklenen bir gizli katman (hidden layer) ve bir çıkış. Gizli katmanda iki nöron, çıkışta bir nöron vardır. Gizli katman sayısı ihtiyaca göre artırılabilir; derin öğrenmenin "derin" olmasının bir sebebi de katman sayısının fazla olmasıdır.

İlk yapay nöron çalışmalarında, insan beyni gibi düşünüldüğü için çok katmanlı ağlar hedefleniyordu. Ama matematiksel işlem çok fazladır. XOR'daki küçük ağda bile girişlerden gizli nöronlara giden ağırlıklar ($v_{11}, v_{12}, v_{21}, v_{22}$), gizli nöronlardan çıkışa giden ağırlıklar (v_3, v_4) ve her nöronun bir eşiği ($\theta_1, \theta_2, \theta_3$) vardır. Gerçek bir sınıflandırma probleminde iki değil çok sayıda özellik ve buna göre çok sayıda ağırlık olur; öğrenme için bunların hepsinin güncellenmesi gerekir. Bu yüzden uzun yıllar MLP'de yalnız bir gizli katman kullanıldı. Zamanla derin öğrenmeyle bu yapı değişti.

Temel öğrenme şöyle olur:

- **İleri yönde:** Girişten başlanarak ileriye doğru çıkış hesaplanır. Birinci gizli nöronda $x_1 v_{11} + x_2 v_{21}$ toplamı θ_1 ile karşılaştırılarak bir çıkış üretilir; ikinci gizli nöronda aynı işlem θ_2 ile yapılır. Gizli nöronların çıkışları çıkış nöronu için giriş olur: Birinci gizli nöronun çıkışı v_3 ile, ikincisinin çıkışı v_4 ile çarpılır ve toplam θ_3 ile karşılaştırılır.
- **Geri yönde:** Çıkışın hatası hesaplanır ve hata çıkış katmanından başlanarak katman katman geriye doğru düzeltilir; ağırlıklar bu hatayı azaltacak şekilde güncellenir.

13.1 Sigmoid Aktivasyon

AND ve OR'da doğrusal bir durum vardı: Girişlerin ağırlıklarla iç çarpımı hesaplanıp eşikle karşılaştırılıyordu. MLP'de doğrusal olmayan bir durum olduğu için aktivasyon fonksiyonu olarak sigmoid kullanılır:

$$y = \frac{1}{1 + e^{-X}}, \qquad X = \sum_{i=1}^{n} x_i v_i - \theta$$

Hoca burada X'in çıkış değil, iç çarpım (eşik çıkarılmış toplam) olduğunu, çıkışın y olduğunu özellikle belirtti. 7. bölümdeki varyasyonlar burada da geçerlidir: Toplam $i = 0$'dan başlatılıp v_0 eşik olarak da yazılabilir. Sonuçta y çıkışı sigmoid fonksiyonuyla hesaplanır.

> **Ek bilgi (derste anlatılmadı):** XOR'un iki katmanla neden çözülebildiğini step fonksiyonlu küçük bir örnek gösterir. Birinci gizli nöron OR gibi çalışsın ($v = 1, 1$, $\theta = 0{,}5$), ikincisi AND gibi ($v = 1, 1$, $\theta = 1{,}5$). Çıkış nöronu "birinci gizli nöron 1, ikincisi 0" durumunu yakalasın ($v_3 = 1$, $v_4 = -1$, $\theta_3 = 0{,}5$). Dört giriş için çıkışlar 0, 1, 1, 0 olur, yani XOR (python ile kontrol edildi). Her gizli nöron bir doğru çizer; çıkış nöronu bu iki doğrunun arasındaki bandı seçer.

14. Geri Yayılım (Backpropagation) Algoritması

Hoca bu kısmı slaytlar üzerinden anlattı. Slaytlarda ciddi örnekler olduğunu, bunları mutlaka paylaşacağını ve öğrencilerin elinin altında olmasını istediğini söyledi.

MLP'deki öğrenme yöntemine geri yayılım (backpropagation) denir. Adını, hatanın geriye doğru yayılarak (propagate) çözülmesinden alır: Hata, çıkış katmanından giriş katmanına doğru katman katman geriye giderek düzeltilir. Önce çıkışa gelen ağırlıklar, sonra girişten gizli katmana gelen ağırlıklar değiştirilir.

Yapıda bir giriş katmanı, bir gizli katman ve bir çıkış katmanı vardır. Giriş katmanındaki bütün girişler gizli katmandaki bütün düğümlere bağlıdır. Gizli katmandaki bütün düğümler de çıkış katmanındaki bütün düğümlere bağlıdır.

14.1 Çıkış Nöronu Sayısı

Hoca öğrencilere şunu sordu: Elle yazılmış 0'dan 9'a kadar rakamları sınıflandırmak istiyoruz. Çıkışta kaç nöron olmalı?

Öğrencilerin cevabı 10'du. Hoca doğru olduğunu söyledi: 10 sınıf vardır ve 0 için bir çıkış nöronu aktif olur, diğerleri 0 olur; 1 için başka bir nöron aktif olur, vb. Sonra şunu sordu: Biraz daha iyileştirmek isterseniz kaç nöronla 10 farklı çıkışı gösterebilirsiniz? Cevap 4'tür, çünkü $2^4 = 16 \geq 10$. Rakamları ikili (binary) olarak kodlarsak 4 çıkış nöronu yeter: 0000 sıfırı, 0001 biri, 0010 ikiyi gösterir. Hem 4 hem 10 nöronla gerçekleştirilebilir. Sonuçta çıkıştaki nöron sayısı sınıf sayısına bağlı olarak verilmesi gereken bir bilgidir.

14.2 Algoritmanın Adımları

Adım 1: Başlatma. Ağda artık yalnız girişlere bağlı ağırlıklar yoktur; ara katmanlardaki ağırlıklar da vardır. Hepsi rastgele değerlerle başlatılır. Genelde $-0{,}5$ ile $+0{,}5$ arasında küçük değerler yeterlidir. Slayttaki örnekte aralık şöyle verilmiştir (F_i, i. nörona gelen giriş sayısı):

$$\left(-\frac{2{,}4}{F_i},\ +\frac{2{,}4}{F_i}\right)$$

Örneğin bir nörona iki giriş geliyorsa aralık $(-1{,}2;\ +1{,}2)$ olur.

Adım 2: Aktivasyon (ileri yönde). Bütün katmanlardaki bütün düğümlerin çıkışları hesaplanır. XOR ağında birinci gizli nöron için:

$$X = x_1 v_{11} + x_2 v_{21} - \theta_1, \qquad y = \frac{1}{1 + e^{-X}}$$

İkinci gizli nöron için aynı hesap v_{12}, v_{22} ve θ_2 ile yapılır. Sonra gizli nöronların çıkışları girdi olarak kullanılarak çıkış nöronunun çıkışı hesaplanır.

Adım 3: Ağırlık eğitimi (geri yönde). Ağırlıklar güncellenirken çıkış katmanından başlanır.

(a) Çıkış katmanı. Önce çıkış nöronu k için hata ve hata gradyanı (error gradient) hesaplanır:

$$e_k(t) = y_{d,k}(t) - y_k(t), \qquad \delta_k(t) = y_k(t)\ [1 - y_k(t)]\ e_k(t)$$

Hoca bazı yıllarda bu ifadenin türev alınarak nasıl bulunduğunu da anlattığını, bu yıl yalnız sonucu verdiğini söyledi. $y_k(1 - y_k)$ çarpanı sigmoid'in türevinden gelir. Bu gradyanla çıkışa gelen ağırlıklar düzeltilir:

$$\Delta v_{jk}(t) = \alpha \cdot y_j(t) \cdot \delta_k(t), \qquad v_{jk}(t+1) = v_{jk}(t) + \Delta v_{jk}(t)$$

Hoca öğrencilere y_j'nin ne olduğunu sordu. Cevap: gizli katmandan gelen değerdir. Hoca bunu çizerek gösterdi: v_{1k} ve v_{2k}, gizli nöronlardan çıkış nöronuna giden ağırlıklardır; y_1 ve y_2 bu gizli nöronların çıkışlarıdır. y_j, çıkış nöronunun girişidir; bir önceki katmanın çıkışıdır.

(b) Gizli katman. Sonra gizli nöron j için hata gradyanı hesaplanır:

$$\delta_j(t) = y_j(t)\ \left[1 - y_j(t)\right] \sum_{k=1}^{l} \delta_k(t)\, v_{jk}(t)$$

Burada l çıkış katmanındaki nöron sayısıdır, v_{jk} ise gizli nörondan çıkış katmanına giden ağırlıklardır. Bu gradyanla girişten gizli nörona gelen ağırlıklar güncellenir:

$$\Delta v_{ij}(t) = \alpha \cdot x_i(t) \cdot \delta_j(t), \qquad v_{ij}(t+1) = v_{ij}(t) + \Delta v_{ij}(t)$$

XOR ağında bu adımda $v_{11}, v_{12}, v_{21}, v_{22}$ güncellenir. Hocanın dikkat çektiği nokta şudur: Gizli katmanın hata gradyanı hesaplanırken çıkıştan gelen hata kullanılır. Gizli nöronun kendi istenen çıkışı yoktur; hatası, çıkış nöronlarının hata gradyanlarının o nöronun ağırlıklarıyla çarpılıp toplanmasıyla bulunur.

Adım 4: İterasyon. Ağırlıklar güncellendikten sonra yakınsama sağlanana, yani hata belirlenen eşiğin altına düşene ya da hata kalmayana kadar işleme devam edilir.

> **Ek bilgi (derste anlatılmadı):** Tek bir güncellemeyi sayılarla görmek için XOR ağında şu değerleri alalım (bu değerler derste söylenmedi, örnek için seçildi): $v_{11} = 0{,}5$, $v_{21} = 0{,}4$, $\theta_1 = 0{,}8$; $v_{12} = 0{,}9$, $v_{22} = 1{,}0$, $\theta_2 = -0{,}1$; $v_3 = -1{,}2$, $v_4 = 1{,}1$, $\theta_3 = 0{,}3$; $\alpha = 0{,}1$. Giriş $(1, 1)$, istenen çıkış 0. İleri yönde gizli çıkışlar $y_1 = \sigma(0{,}1) \approx 0{,}525$ ve $y_2 = \sigma(2{,}0) \approx 0{,}881$, ağ çıkışı $y \approx 0{,}510$ olur. Hata $e \approx -0{,}510$, çıkış gradyanı $\delta \approx 0{,}510 \cdot 0{,}490 \cdot (-0{,}510) \approx -0{,}127$. Çıkış ağırlıkları $\Delta v_3 \approx 0{,}1 \cdot 0{,}525 \cdot (-0{,}127) \approx -0{,}0067$ ve $\Delta v_4 \approx -0{,}0112$ kadar değişir. Gizli gradyanlar $\delta_1 \approx 0{,}038$ ve $\delta_2 \approx -0{,}015$ olur. Değerler python ile hesaplandı; Δv_3'ün hata fonksiyonunun sayısal türeviyle aynı yönde ve büyüklükte olduğu da kontrol edildi.

15. XOR'un Öğrenilmesi, Momentum ve Hiperbolik Tanjant

Hoca slaytlarda örnek ağırlık değerleriyle XOR'un nasıl öğrenildiğinin adım adım gösterildiğini söyledi. Örneğin üçüncü nöronun (birinci gizli nöron) çıkışı $y_3 = 1/\left(1 + e^{-(x_1 v_{13} + x_2 v_{23} - \theta_3)}\right)$ ile, dördüncü nöronunki de benzer biçimde hesaplanıyor. Hoca bu hesapları derste tek tek okumanın anlamı olmadığını söyledi.

> **Hocanın vurgusu:** Bir tarafa ağı elle çizin, bir tarafa slaytları alın ve y_1, y_2, y_3'ün nasıl hesaplandığını tek tek kendiniz görün. Biraz karışık bir konu; bunu görmenizi istiyorum. Slaytlar online kampüste paylaşılacak.

Hoca slayttaki sonuçları şöyle aktardı:

- **Hata grafiği:** Epoch sayısı arttıkça hata kareleri toplamının nasıl azaldığı gösteriliyor. Eğitim başında hata 10^0 ile 10^1 arasındadır (yaklaşık 3). Önce 1'e iner, sonra daha da azalır. Yaklaşık 200 epoch'tan sonra 10^{-3}'e düşer; hoca slayttan 224 epoch'ta öğrendiğini okudu.
- **Son çıkışlar:** Sigmoid kullanıldığı için çıkışlar artık 0 ya da 1 değil, ondalıklı sayılardır. Son aşamada her giriş için gerçek çıkışlar ve hatalar tabloda verilir; hata belirlenen değerin altına düşünce eğitim kesilmiştir.
- **Öğrenilmiş ağırlıklar:** Ağ XOR'u öğrendiğinde ulaşılan ağırlıklar slaytta verilmiştir.
- **Karar sınırları:** (a) üçüncü nöronun (birinci gizli nöron) karar sınırı, (b) dördüncü nöronun karar sınırı, (c) ikisinin birleşimi. Çıkıştaki durum, gizli nöronların karar sınırlarının birleşmesiyle oluşur.

15.1 Daha Hızlı Öğrenme

Hoca öğrenmeyi hızlandırmak için iki yöntemden söz etti.

- **Hiperbolik tanjant:** Sigmoid fonksiyonu yerine hiperbolik tanjant (hyperbolic tangent) kullanılabilir. Slayttaki formülde a ve b sabittir.
- **Momentum:** Ağırlık güncellemesine bir momentum terimi eklenebilir. Öğrenme oranı küçükse ağ yavaş öğrenir; momentum eklendiğinde öğrenme oranı da daha büyük alınabilir. Momentum 0 ile 1 arasında bir değerdir, genellikle 0,9 civarında alınır. Hoca momentum için 0,9, öğrenme oranı için 0,5 gibi değerlerle eğitim yapılabileceğini söyledi.

Slayttaki karşılaştırmaya göre momentumsuz ağ 200'ü geçip 224 epoch'ta öğrenmişti. Momentum eklendiğinde hata yaklaşık 120 epoch'ta 10^{-3}'e iniyor. Momentum katsayısı ekleyerek ağın daha hızlı öğrenmesi sağlanabilir.

> **Ek bilgi (derste anlatılmadı):** Momentumlu güncelleme kuralı (genelleştirilmiş delta kuralı) genelde şöyle yazılır: $\Delta v_{jk}(t) = \beta \cdot \Delta v_{jk}(t-1) + \alpha \cdot y_j(t) \cdot \delta_k(t)$. Burada β momentum katsayısıdır. Bir önceki adımdaki değişimin bir kısmı korunur; böylece ağırlıklar aynı yönde ilerlerken hızlanır, yön değiştirirken salınım azalır. Kaynaklarda yaygın kullanılan hiperbolik tanjant biçimi $y = \dfrac{2a}{1 + e^{-bX}} - a$'dır; çıkış $-a$ ile $+a$ arasında değişir.

Dersin Toparlaması

Bu derste biyolojik nörondan yola çıkarak yapay nöron ağlarına giriş yaptık. Biyolojik nöronda dendritler girişleri toplar, sinapslar her girişin etkisini belirler ve ağırlıklı toplam bir eşiği aşarsa akson üzerinden sinyal iletilir. Yapay nöron bunu girişler, ağırlıklar, bir toplayıcı ve bir aktivasyon fonksiyonuyla taklit eder. En basit model olan perceptron, ağırlıklı toplamı bir eşikle karşılaştırarak iki sınıflı bir karar verir ve sınıfları bir hiper düzlemle doğrusal olarak ayırır.

Perceptron'un öğrenmesi tamamen ağırlıkların değişmesidir. Delta kuralı, istenen ile hesaplanan çıkış arasındaki hatayı, öğrenme oranını ve girişi kullanarak ağırlıkları günceller; hata pozitifse ağırlık artar, negatifse azalır. AND ve OR örneklerinde bu kuralın adım adım nasıl uygulandığını ve bir epoch boyunca hiç hata kalmayınca öğrenmenin bittiğini gördük. Eşiğin

ayrı tutulması, sabit bırakılması ya da x_0 girişine bağlı bir ağırlık olarak yazılması gibi varyasyonların hepsi aynı sonuca çıkar.

XOR problemi tek katmanlı perceptron'un sınırını gösterdi: XOR'un sınıfları doğrusal olarak ayrılamaz. Çözüm, girişle çıkış arasına gizli katman eklenen çok katmanlı perceptron'dur. MLP sigmoid aktivasyon kullanır ve geri yayılım algoritmasıyla öğrenir: Çıkış ileri yönde hesaplanır, hata çıkış katmanından başlayarak geriye doğru yayılır; önce çıkışa giden, sonra gizli katmana giden ağırlıklar güncellenir. Momentum ve hiperbolik tanjant bu öğrenmeyi hızlandırmanın iki yoludur. Hoca XOR geri yayılım örneğinin slaytlarını paylaşacağını ve bu hesapların elle tek tek takip edilmesini istediğini söyledi.

Ders Sonrası Notlar

Ders sonrası not: Hoca sigmoid için "çıkışta 0, 1 değil, negatif veya pozitif bir değer üretmiş oluyorsunuz" dedi. Sigmoid'in çıkışı her zaman 0 ile 1 arasındadır, negatif değer üretmez. Kastedilen, çıkışın yalnız 0 ve 1 olmayıp aradaki sürekli değerleri alabilmesidir. Negatif ve pozitif değer üreten yumuşak fonksiyon hiperbolik tanjanttır (-1 ile $+1$ arası).

Ders sonrası not: AND örneğinde hoca hatayı bir yerde "hesaplanan çıkış çarpı (gerçek çıkış eksi hesaplanan çıkış)" diye söyledi. Doğrusu yalnızca $e = y_d - y$'dir; hesaplanan çıkışla çarpım yoktur. Ağırlık değişimi $\Delta v_i = \alpha \cdot e \cdot x_i$'dir. (Çıkışla çarpım, $y(1-y)$ biçiminde, yalnızca sigmoid kullanan geri yayılımdaki hata gradyanında görülür.)

Ders sonrası not: Hoca perceptron öğrenme kuralını hatanın türevinden, yani gradyan inişinden türetti. Bu türetme çıkışın doğrusal olduğu durum (Widrow-Hoff/ADALINE) için tam olarak geçerlidir. Step ve sign fonksiyonlarının türevi sıfır ya da tanımsızdır; bu yüzden perceptron kuralı kesin anlamda bir gradyan inişi değildir, ama aynı biçimdeki güncelleme kuralını kullanır. Rosenblatt'ın perceptron kuralı, sınıflar doğrusal ayrılabiliyorsa sonlu adımda yakınsar.

Ders sonrası not: Hoca yapay nöron çalışmalarının "1940'ların sonlarında" başladığını söyledi. McCulloch ve Pitts'in yapay nöron modeli 1943 yılına aittir; Rosenblatt'ın perceptron'u 1958'dedir.

Ders sonrası not: Excel'de algoritmayı kuran öğrencinin sorusunda karar sınırı $0{,}1x_1 + 0{,}1x_2 = 0$ diye yazıldı. Eşik dahil doğru karar sınırı $0{,}1x_1 + 0{,}1x_2 - 0{,}2 = 0$, yani $x_1 + x_2 = 2$'dir (9. bölümdeki ek bilgi).

Ders sonrası not: Hoca XOR tablosunu anlatırken bir an "1-1 için 1 istiyorsunuz" dedi, hemen ardından düzeltti. XOR'da $(1, 1)$ girişinin çıkışı 0'dır.

Ders sonrası not: Altyazıdaki bozulmalar: "Mark Klok ve Pitts" McCulloch ve Pitts, "Hofloin'in rolü" Widrow-Hoff kuralı, "Deltorum" delta rule, "Dat product" dot product, "egzor" XOR, "ent" AND, "ipok" epoch, "trash/treş/tıraş olduğu" threshold, "Bayes" (bias sorusunda) bias, "time function" ve "side function" sign function, "shape fonksiyonu" step fonksiyonu, "sigmoik/sıkı poit" sigmoid, "Monteneyat/Matta-değer/maltı" multi-layer, "hypopalic tangent" hyperbolic tangent, "iPhone" ve "burning rate" ise öğrenme oranı (α) ya da learning rate'tir. "Kormojeniz" convergence (yakınsama), "Outlook Layer/kutlayır" output layer demektir. Slaytta başlangıç aralığındaki "x2.4" ifadesi $2{,}4/F_i$'dir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.